

NanoInfer 推理引擎 功能全景

NanoInfer

2026-08-16

C++ + CUDA 手写 LLM 推理引擎，对标 vLLM / SGLang，零 Python 推理依赖。
核心能力：**PagedAttention** + 前缀缓存 + **Continuous Batching**，逐 token 复现 HF 输出
(Qwen2.5 0.5B / 1.5B / 3B / 7B 对拍“Paris”一致)。

依赖方向:ops → kv_cache → engine → server,静态库链接 CUDA::cudart + CUDA::cublas。

1 模型加载与表示

- **safetensors** 解析 (model/model_loader): 单文件 + 分片 model.safetensors.index.json (3B / 7B 分片), F32 / BF16 / FP16; BF16 用 uint16<<16 重解释为 float。
- **ModelConfig**: 解析 HF config.json, 支持 Llama / Qwen2 (结构同构); GQA kv_groups()、rope_theta、tie_word_embeddings。
- **FP8 weight-only** 加载: 权重 F16 / BF16 → E4M3 + per-row scale (DType::FP8_E4M3)。
- **untied lm_head** 支持: lm_head_weight() = lm_head_w_.defined() ? lm_head_w_ : embed_w_ (7B 关键, tie_word_embeddings=false)。
- **Tensor**: RAII, F32 / F16 / BF16 / I32 / FP8_E4M3 五种 dtype, CPU / CUDA; view() (不拥有数据) + reshape_inplace() (修复自赋值悬垂)。

2 算子层 ops (13 个源文件)

类别	内容
GEMM	5 级后端 CuBLAS / Naive / Tiled / Vectorized / DoubleBuf (430→1646 GFLOPS); gemm_f16f32 激活 F16 × 权重 F16 → F32 (Tensor Core, FP32 累加)。
FP8	quantize_fp8; gemm_f16_fp8 (朴素); gemm_f16_fp8_cublas (路线 A, W8A16 反量化 + cuBLAS); gemm_f16_fp8_w8a8 (路线 B, 真 FP8 TC + 探测自动回退); gemm_f16_fp8_opt (自适应 RM + 向量化 + 双缓冲); gemv_f16_fp8 (M=1 split-K 专用); fp8_w8a8_available() (Hopper-only 探测)。
注意力	attention 朴素版 (GQA 支持, 正确性基线); PagedAttention decode / prefill 两套 kernel (见 §3)。

类别	内容
其它	RMSNorm / RoPE (rotate_half) / softmax / embedding / lm_head / reduce / elementwise / activation / argmax / unary。
采样 (CPU 端)	greedy / temperature / top-k (O(n) nth_element) / top-p / min-p / multinomial; <code>sample_batch</code> 批量独立采样。

3 KV Cache / PagedAttention (kv_cache/)

- **BlockAllocator**: 16 token/block ([block_size, Hkv, head_dim]), 多序列共享块 (ref_count), KV dtype F32 / F16。
- **BlockTable**: 逻辑 → 物理块映射; 全层锁步分配 / 释放。
- **decode kernel**: 每 (token, Q-head) 一个 block, Float4 向量化 + DoubleBuf + Split-K。
- **prefill kernel**: 直接按 block table 读 KV, 无需重建连续张量; 按 P/D 自动选 kernel (P≤64 direct、P>64 tiled Flash variant); causal 掩码可选。
- **GPU 散射**: scatter_prefill_kv_gpu 单 kernel 替代逐 token memcpy; first_prefill_attn_batched_gpu 一次 launch 处理 N 请求。
- **前缀缓存** (env NANOINFER_PREFIX_CACHE=hash|radix):
 - HashPrefixCache: 单 block 哈希 (vLLM 风格) + LRU。
 - RadixPrefixCache: SGLang 风格 Patricia trie——节点分裂、ref_count 共享、LRU 淘汰, 跨块最长前缀匹配。

4 推理引擎 engine

- **EngineServer**: step(ScheduleStep) 消费异构 batch (prefill chunk + decode token 混在一个 forward), 返回 SampledToken。
- **RequestState**: 每请求持久状态——逐层 block_tables、next_hidden_state、num_cached_tokens (前缀命中)、pending_first_token、grammar_matcher、投机统计。
- **请求生命周期**: create_request_state → add_request → 循环 step() → release_request (释放全部层 KV block)。
- **Scheduler**: Chunked Prefill (CHUNK=16) + 4 策略 (FCFS / PrefillFirst / DecodeFirst / Interleave, 默认 DecodeFirst); max_batch_tokens 预算。
- **多前向路径**: forward_n_layers (纯计算) / forward_partial / forward_batched (投机验证) / forward_incremental (写 KV); run_layers 从任意层开始, draft 引擎传自己的 allocators 不污染 target KV。
- **BatchMainLoop**: continuous batching 编排, 产出 TTFT / TPOT / percentile(p50/p90/p95/p99) / throughput 指标 + 每步 batch 效率。

5 投机解码 (speculative_decoder + draft_engine)

- **双策略**: SelfSpecDraftEngine (target 前 M 层作草稿, 自己的 KV cache, 权重共享只读); DualModelDraftEngine (独立小模型, 自己的 EngineServer)。
- **两种验收**: strict argmax (默认, 输出与 greedy 逐字一致); Leviathan 概率接受 (输出匹配 target 分布, 跨尺度 dual 模型必需)。
- **verify_batch 开关**: 批式 M=k 验证 vs 顺序 M=1 (数值一致性权衡)。
- **SpeculativeStats**: drafted / accepted / rejected / 各阶段耗时统计。
- **实测现状**: 修复 RoPE bug 后重测为净损失 (旧 1.2–1.55× 是 bug 假象), 当前不作为性能主线。详见《投机解码技术开发手册》。

6 结构化输出 (xgrammar 集成)

- JSON Schema / Regex / EBNF / GBNF → BNF → NFA → **DFA** → CompiledGrammar (LRU 缓存)。
- **每步**: FillNextTokenBitmask() (int32 位掩码) → 置 $-\infty$ → 采样 → AcceptToken() 推进状态。
- **AdaptiveTokenMask** 三分类 (约 97% 编译期预计算, 运行时查表), 开销 < 0.1 ms/step; 140k 断言测试覆盖。

7 HTTP 服务 / Tokenizer

- **HTTP**: 单线程 blocking server (localhost:8080, JSON, /v1/chat/completions 风格 handler)。
- **Tokenizer**: HF tokenizer.json 加载, decode 完整; encode 为简化版 (生产建议 HF Python 预分词); 字节级 BPE, merge_rank_ O(1) 查找 (修过 15 万 merges 线性扫描卡顿)。

8 测试与基准工具

- 约 30 个 **test_***: tensor / fp16 / ops / quality / model / e2e / full_model / paged_attention(×2) / attention_fp16 / paged_prefill / paged_model / generate / sampling / structured_output / structured_e2e / lm_head / concurrent / scheduler / tokenizer / server_engine / batch_loop / prefix_cache / radix_engine / mixed_users / spec_decode / verify_incremental / attention_divergence ...
- **tools**: quick_generate、bench_server / simulation / throughput / fp16 / concurrency / prompt_lengths / spec_decode / max_users、bench_fp8_gemm、bench_batch_quality、test_fp8、dump_weight。

9 精度路径与已验证数据

- 精度路径：FP32 默认；FP16 激活（GEMM / norm / elementwise 用 F16，*attention* 保持 F32）；FP8 weight-only。
- 正确性：4 模型逐 token 复现 HF（11 个前向 bug 全修）；增量解码与全量重算 `max_err=0`。
- 性能基线（RTX 4090）：7B fp16 单请求 TPOT $\approx 25\text{ms}$ 、5 并发 $\approx 214\text{tok/s}$ ；FP8 路线 A +10%~72% 吞吐。